
OreSat

Resumen ejecutivo para el equipo INTISAT

Que es, como esta construido, y que podemos robar de su arquitectura

Origen: Portland State Aerospace Society (PSAS)
Universidad: Portland State University, Oregon, USA
Filosofia: 100 % open-source (HW + FW + SW + GS)
GitHub: <https://github.com/oresat>
Audiencia: Lideres tecnicos del proyecto INTISAT
Documento: v1.0
Fecha: 12 de mayo de 2026

Resumen. OreSat es uno de los CubeSats academicos mas documentados y abiertos del mundo. Construido por estudiantes de Portland State University, su arquitectura modular por *cards* comunicadas via CAN bus, su documentacion publica, y su iteracion entre misiones (OreSat0, 0.5, 1) lo hacen un caso de estudio ideal para el INTISAT. Este resumen identifica las practicas mas valiosas que el equipo deberia adoptar, los repositorios mas relevantes para revisar, y la comparacion directa contra el estado actual del INTISAT.

Índice general

1. Que es OreSat	2
1.1. Origen y mision	2
1.2. La filosofia del open source	2
1.3. Numeros del programa	2
2. Las misiones	3
2.1. OreSat0 (2022): el primer vuelo	3
2.2. OreSat0.5: iteracion intermedia	3
2.3. OreSat1: la mision cientifica	3
3. Arquitectura modular — las "cards"	4
3.1. El concepto de card	4
3.2. Las cards tipicas	4
3.3. La interconexion: CAN bus	4
3.4. Por que esta arquitectura es buena	4
4. Stack de software	5
4.1. Tres niveles	5
4.2. El C3 (Command, Control, Communications)	5
4.3. Variantes con NASA cFS	5
5. Estructura de repositorios en GitHub	6
5.1. Prioridad alta para INTISAT	6
5.2. Prioridad media	6
5.3. Prioridad baja (pero interesante)	6
6. Lecciones aplicables al INTISAT	7
6.1. 1. Documentacion como first-class deliverable	7
6.2. 2. Arquitectura modular por cards	7
6.3. 3. CAN bus + CANOpen para comunicacion entre cards	7
6.4. 4. Iteracion entre misiones	7
6.5. 5. Open source desde el dia 0	7
6.6. 6. Test-driven development para hardware embebido	8
6.7. 7. Ground station propia y simple	8
7. Comparacion directa: OreSat vs INTISAT	9
7.1. Donde estamos mejor que OreSat	9
7.2. Donde estamos atrasados	9
8. Plan de exploracion sugerido	10
8.1. Output esperado al final de la semana	10
9. Conclusion y proximos pasos	11
9.1. Enlaces principales	11

1. Que es OreSat

1.1. Origen y mision

OreSat es el programa de CubeSats de **Portland State Aerospace Society (PSAS)**, un grupo estudiantil de Portland State University (Oregon, EEUU). El programa nacio en 2017 con un proposito doble:

- Hacer ciencia espacial real (mediciones climaticas en LEO).
- Educar a estudiantes en ingenieria aeroespacial.

1.2. La filosofia del open source

Lo que diferencia a OreSat de otros CubeSats academicos:

- Todo el **hardware** (esquemáticos, layouts, BOMs) en GitHub.
- Todo el **firmware** (codigo de cada subsistema) en GitHub.
- Todo el **flight software** (OBC + apps) en GitHub.
- Todo el **ground station** (software, hardware, antenas) en GitHub.
- **Documentacion exhaustiva**: design reviews, lessons learned, test procedures.

Esto es raro. La mayoría de los CubeSats academicos liberan al menos algunos componentes propietarios. OreSat libera *todo*, en el espiritu de la *Libre Space Foundation* con la que tienen alianza tecnica.

1.3. Numeros del programa

Atributo	Valor
Año de inicio	2017
Misiones lanzadas	OreSat0 (2022)
Misiones en desarrollo	OreSat0.5, OreSat1
Tamaño del equipo	30-50 estudiantes activos
Tiempo de vuelo OreSat0	14 meses (deorbit enero 2024)
Repos en GitHub	30 activos
Estandares aplicados	CalPoly CDS, ISO 9001

2. Las misiones

2.1. OreSat0 (2022): el primer vuelo

Mision **1U** (un cubo de 10 cm) lanzada en mayo 2022 desde la ISS via Nanoracks deployer. Objetivos:

- Validar la arquitectura por cards en orbita real.
- Probar el OBC con Linux.
- Probar las comms UHF.
- Demostrar autonomia operativa.

Resultado: el satellite funciono nominalmente durante 14 meses antes de quemarse en reentrada (enero 2024). Confirmo la viabilidad del diseño.

2.2. OreSat0.5: iteracion intermedia

Version mejorada del OreSat0 con correcciones aprendidas. En desarrollo al momento de redactar este documento. Mismo tamaño 1U.

2.3. OreSat1: la mision cientifica

CubeSat **2U** con payload cientifico principal: un imager de **cirros (nubes altas)** para climatologia. Tambien lleva DXWiFi (downlink de imagenes via WiFi modificado) y un livestream payload.

Leccion aplicable al INTISAT

OreSat siguio el patron *primero validacion tecnica, despues mision cientifica*. INTISAT podria adoptar el mismo: una mision corta validatoria, seguida de la mision con payload de microscopia.

3. Arquitectura modular — las "cards"

3.1. El concepto de card

OreSat se construye apilando **cards** (placas PCB) en formato PC-104. Cada card es un subsistema independiente con:

- Su propio microcontrolador.
- Su propio firmware.
- Su propio conector al backplane CAN.

3.2. Las cards típicas

Card	Funcion	MCU
C3 (Command, Control, Comms)	OBC principal, Linux, comms UHF	ARM A9
Solar	generacion solar + MPPT	STM32
Battery	gestion de bateria	STM32
Power	distribucion + reguladores	STM32
ADCS	control de actitud	STM32
Imager	camara payload	STM32
DXWiFi	downlink por WiFi modificado	STM32
StarTracker	determinacion fina de actitud	STM32

3.3. La interconexion: CAN bus

Todas las cards se comunican por un bus **CAN** compartido en el backplane PC-104. Usan el protocolo **CANopen**, estandar industrial.

Leccion aplicable al INTISAT

Tu INTISAT usa I2C primario hacia el OBC. Si en el futuro tenes mas cards (múltiples sensores, subsystem propio para FPM), CAN+CANopen es el camino que OreSat ya valido. Ahorras tiempo y errores tipicos.

3.4. Por que esta arquitectura es buena

- **Cards intercambiables:** si falla la Battery card, se reemplaza sin afectar al resto.
- **Desarrollo paralelo:** cada equipo trabaja en su card independientemente.
- **Testing aislado:** cada card se testea sola en banco antes de integrar.
- **Reusabilidad entre misiones:** la Solar card de OreSat0 se reuso en OreSat1.
- **Tolerancia a fallos:** una card en falla no contamina el bus CAN gracias a CANopen.

4. Stack de software

4.1. Tres niveles

OreSat tiene 3 capas de software:

- **Firmware de cards:** bare-metal C o ChibiOS RTOS en STM32. Hace lo basico (sensar, actuar, hablar CAN).
- **Flight software del OBC (C3):** Linux con servicios en Python. Coordina las cards, maneja comms, ejecuta logica de mision.
- **Ground station:** Python (oresat-gs), procesa telemetria y envia comandos.

4.2. El C3 (Command, Control, Communications)

Es el OBC principal de OreSat. Caracteristicas:

- Linux Debian custom.
- Servicios Python: `oresat_c3_software`.
- Comunicacion con cards via socketCAN.
- Comms con tierra via UHF (modulo Si4463).
- Almacenamiento: eMMC.

Leccion aplicable al INTISAT

Es exactamente lo que tu `cubesat/daemon.py` es para el payload, pero a escala de satelite completo. Vale leer el codigo del C3 para ver como organizan los servicios Python en un OBC operativo.

4.3. Variantes con NASA cFS

OreSat tambien explora la integracion con **NASA cFS** (Core Flight System) en el repo `oresat0-cfs`. Esto es valioso para entender como adaptar el framework profesional de NASA al hardware academico.

5. Estructura de repositorios en GitHub

Los 30+ repos de OreSat estan en <https://github.com/oresat>. Los mas valiosos para revisar (por orden de prioridad):

5.1. Prioridad alta para INTISAT

- **oresat-c3-software**: el flight software del OBC.
Por que mirarlo: tu daemon hace algo similar para el payload. Ver como organizan ellos un OBC entero.
- **oresat-cans**: definiciones del protocolo CAN.
Por que: si en el futuro agregas CAN como backup, esta es la plantilla.
- **oresat-firmware**: firmware de cada card.
Por que: ejemplos de como estructurar firmware embebido portable y testeable.
- **oresat-mainboard**: PCB del backplane.
Por que: como hacer un backplane PC-104 con CAN.

5.2. Prioridad media

- **oresat-configs**: configuraciones de mision.
- **oresat-launch-services**: software ground station.
- **oresat0-imager**: payload de camara (similar concepto al del INTISAT pero EO en vez de microscopia).
- **oresat0-cfs**: integracion con NASA cFS.

5.3. Prioridad baja (pero interesante)

- **oresat-pre-launch**: documentacion de las reviews.
- **oresat0-dxwifi**: downlink de imagenes via WiFi modificado (curioso pero no aplica directo).
- **oresat-livestream**: video downlink en orbita.

Cuidado / verificar

La lista de repos puede haber cambiado desde la redaccion de este documento. Verificar en <https://github.com/oresat> para el listado actualizado.

6. Lecciones aplicables al INTISAT

Las practicas que mas vale la pena copiar:

6.1. 1. Documentacion como first-class deliverable

OreSat publica TODO. Cada decision de diseño, cada lesson learned, cada review tecnica. Eso les permite:

- Onboardear miembros nuevos rapido.
- Recibir contribuciones externas.
- Justificar decisiones ante revisores.
- Reusar conocimiento entre misiones.

Que adoptar: tu Documentos de Referencia/ esta bien encaminado. Falta capturar las *decisiones de diseño* (por que elegimos X y no Y) y las *lessons learned* (que aprendimos cuando fallo Z).

6.2. 2. Arquitectura modular por cards

Cada subsistema es independiente. Esto reduce la complejidad y permite desarrollo paralelo.

Que adoptar: en el INTISAT, separar claramente los subsistemas del payload. Tu payload ya esta bien aislado (carpeta `cubesat/` con interfaz I2C clara). Mantener esa separacion estricta.

6.3. 3. CAN bus + CANopen para comunicacion entre cards

OreSat eligio CAN porque es robusto, estandarizado, y soporta multi-master naturalmente.

Que adoptar: cuando el INTISAT pase de OBC simple a multi-subsystem, considerar CAN como bus primario (no I2C). I2C es mas simple pero menos robusto a EMI.

6.4. 4. Iteracion entre misiones

OreSat0 valida → OreSat0.5 mejora → OreSat1 hace ciencia. Cada mision aprende de la anterior.

Que adoptar: planear el INTISAT como una serie, no como mision unica. Primera mision: validar payload de microscopia con muestra simple. Segunda mision: agregar funcionalidades (FPM completo, mas patrones de OLED).

6.5. 5. Open source desde el dia 0

OreSat NO empezo cerrado y luego abrio. Empezo abierto. Esto es una disciplina dificil pero rinde:

- Codigo mas limpio (saben que lo veran extraños).
- Pull requests externos.
- Visibilidad academica.

- Reproducibilidad.

Que adoptar: tu repo CubeSat-EdgeAI-Payload ya es publico. Falta agregar:

- Un LICENSE explícito (MIT o Apache 2.0).
- Un CONTRIBUTING.md con guidelines.
- Issues activos para que externos puedan contribuir.
- Código de conducta.

6.6. 6. Test-driven development para hardware embebido

OreSat tiene tests automatizados de firmware (unit + integration). Esto es raro en CubeSat académico pero crucial para confianza.

Que adoptar: tu daemon en Python debería tener tests con pytest. Especialmente para el dispatcher del I2C slave y para el OTA.

6.7. 7. Ground station propia y simple

OreSat tiene su ground station en Portland. No depende de servicios de terceros. Software Python fácil de modificar.

Que adoptar: planear desde temprano una ground station académica (antena UHF + SDR + Python con SatNOGS). No esperar al último momento.

7. Comparacion directa: OreSat vs INTISAT

Aspecto	OreSat	INTISAT (actual)
Tamaño	1U (OreSat0), 2U (OreSat1)	3U (planeado)
Filosofia	100 % open source	open en GitHub, no formalizado
OBC	C3 card con Linux Debian	Pi 5 (CM5 vuelo) con Pi OS
Buses internos	CAN + CANopen	I2C (CAN como backup futuro)
Payload Cards	Imager EO cirrus 7-8 cards independientes	Microscopia FPM lensless 1 card del payload + OBC central (separados)
Ground station	propia, Python, en Portland	no implementada aun
Misiones planeadas	OreSat0/0.5/1, iterativo	INTISAT (1 mision por ahora)
Documentacion publica	exhaustiva, en GitHub	creciendo, en Documentos de Referencia/
NASA cFS	explorado en oresat0-cfs	no usado (custom daemon)
Equipo	30-50 estudiantes	4 personas

7.1. Donde estamos mejor que OreSat

Pocas cosas, pero existen:

- **Payload diferenciado:** microscopia biologica lensless es un nicho menos explotado que EO de cirros.
- **IA on-board:** usamos StarDist y Cellpose para procesar a bordo. OreSat solo captura.
- **Pipeline completo:** tenemos preprocesado, mejora con IA, segmentacion y exportacion en un solo daemon.

7.2. Donde estamos atrasados

- **Tamaño del equipo:** ellos tienen 30-50, nosotros 4.
- **Misiones lanzadas:** ellos ya volaron 1, nosotros 0.
- **Modularidad:** ellos tienen 7 cards, nosotros 1.
- **Ground station:** ellos tienen, nosotros no.
- **Test coverage:** ellos automatizan, nosotros no.

8. Plan de exploracion sugerido

Para el equipo del INTISAT (4 personas), proponemos dividir la investigacion de OreSat en 1 semana:

Dia	Repo / tema	Que extraer
1	oresat-c3-software	arquitectura del OBC con Linux: como dividen servicios, como manejan estados
2	oresat-cans oresat-firmware	+ como definen CAN messages y los mapean a firmware
3	oresat0-imager	comparar con tu payload de microscopia: que es comun, que es distinto
4	oresat-launch-services	inspiracion para tu ground station
5	Documentacion publica (wiki / docs)	lessons learned

8.1. Output esperado al final de la semana

- Lista de 5-10 practicas concretas que adoptar en INTISAT.
- Lista de 3-5 cosas a NO copiar (porque no aplican).
- Borrador de una arquitectura modular para INTISAT v2.

9. Conclusion y proximos pasos

OreSat es probablemente el **mejor caso de estudio publico** para un CubeSat academico operativo. Vale la pena que el equipo de INTISAT lo revise en profundidad.

Recomendacion

Tres acciones concretas para esta semana:

1. Cada miembro del equipo clona un repo de OreSat segun la tabla del Capitulo 7 y lo explora 2 horas.
2. Reunion del equipo (1 hr) para sintetizar: que adoptar, que no.
3. Agregar LICENSE y CONTRIBUTING.md al repositorio del INTISAT (1 hora).

Cuidado / verificar

Este resumen ejecutivo se basa en informacion publica de OreSat al momento de redaccion. Para detalles tecnicos especificos (nombres exactos de repos, version de los protocolos, etc.) verificar siempre contra <https://github.com/oresat> y <https://oresat.org>.

9.1. Enlaces principales

- Organizacion GitHub: <https://github.com/oresat>
- Sitio web: <https://oresat.org>
- Portland State Aerospace Society: <https://psas.pdx.edu>
- Libre Space Foundation (aliados): <https://libre.space>